

Wintap 7 Developer Guide

Table of Contents

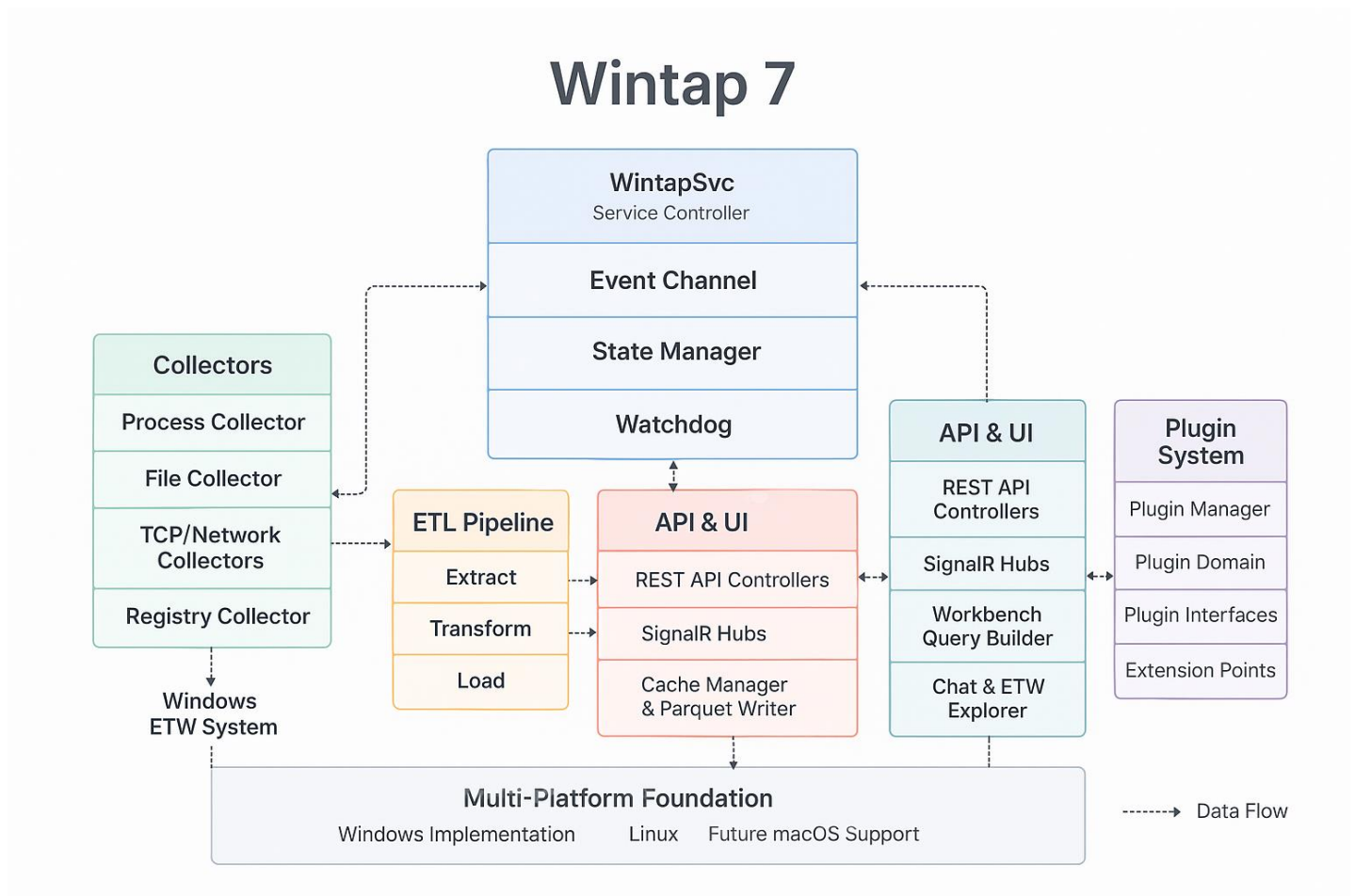
- [Introduction](#)
- [Architecture Overview](#)
- [Core Infrastructure](#)
- [Collector Framework](#)
- [Data Model](#)
- [ETL Pipeline](#)
- [Plugin System](#)
- [Web API and User Interface](#)
- [Query Language](#)
- [Security Considerations](#)
- [Deployment and Configuration](#)
- [Troubleshooting and Diagnostics](#)
- [AI Assistant Integration](#)
- [Multi-Platform Support](#)
- [API Reference](#)
- [Contributing Guidelines](#)
- [Licensing Information](#)

Introduction

Wintap 7 is a sophisticated host-based telemetry and monitoring system designed to collect, process, and analyze system events across multiple platforms, with primary support for Windows and emerging support for Linux and macOS. It provides real-time visibility into process activity, file operations, network connections, registry changes, and other system events through an extensible architecture.

Key Features

- High-performance ETW (Event Tracing for Windows) integration
- Extensible plugin architecture with domain isolation
- Real-time event processing using Esper EPL (Event Processing Language)
- Structured data storage using Parquet format
- Web-based query and visualization interface
- AI-assisted analysis capabilities
- Cross-platform compatibility foundations



System Requirements

- Windows 10/11 or Windows Server 2016+
- .NET 6.0 or higher
- 4GB RAM minimum (8GB recommended)
- 1GB free disk space for application and logs

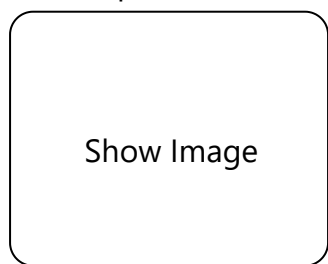
How to Use This Guide

This developer guide is intended for software developers who need to extend Wintap's functionality, integrate it with other systems, or understand its internal architecture. Each section provides both conceptual information and practical implementation details.

Code samples throughout this guide use C# and are compatible with the Wintap 7.0.0.0 API. The guide assumes familiarity with object-oriented programming and basic knowledge of Windows system architecture.

Architecture Overview

Wintap uses a layered architecture organized around several core components that work together to collect, process, and expose system telemetry data.



Show Image

Component Overview

1. **Core Infrastructure:** The central foundation of the system, handling service management, event processing, and state management.
2. **Collectors:** Components responsible for acquiring raw telemetry data from various system sources.
3. **ETL Pipeline:** Processes raw telemetry into structured data for storage and analysis.
4. **Plugin System:** Provides extensibility through isolated plugin domains.
5. **Web API and UI:** Exposes data and control functions through both REST APIs and a web-based UI.

Data Flow

1. **Acquisition:** Collectors gather events from the underlying operating system.
2. **Processing:** Events are normalized into the WintapMessage format and enriched with additional context.
3. **Streaming:** Processed events flow through the Event Channel where they can be consumed by subscribers.
4. **Storage:** Events are optionally persisted to disk in Parquet format.
5. **Query:** Real-time and historical data can be queried using EPL statements.
6. **Visualization:** Results are presented through the web UI or made available via API.

Key Design Patterns

- **Singleton Pattern:** Used for core services like EventChannel, StateManager, and WintapLogger
- **Observer Pattern:** Used for event notification across the system
- **Factory Pattern:** Applied in collector and plugin management
- **Repository Pattern:** Used for data access abstraction
- **Dependency Injection:** Used in the web API layer

Architectural Considerations

Wintap was designed with the following considerations:

- **Performance:** Minimizing overhead on host system performance
- **Reliability:** Ensuring stable operation through Watchdog monitoring
- **Security:** Limiting privileges and access to system resources
- **Extensibility:** Providing clean extension points via the plugin system
- **Cross-Platform:** Establishing abstractions to support multiple operating systems

Core Infrastructure

The core infrastructure comprises the foundational components that support Wintap's operation. These components handle service lifecycle, event processing, state management, and system monitoring.

WintapSvc

The `WintapSvc` class serves as the main entry point and service manager for Wintap. It inherits from `BackgroundService` to integrate with the .NET hosting model.

csharp



```
public partial class WinTapSvc : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken
stoppingToken)    {
        // Initialize core components
        // Start background workers
        // Set up event handlers

        while (!stoppingToken.IsCancellationRequested)
        {
            await Task.Delay(1000,
stoppingToken);
        }
    }

    public override async Task StopAsync(CancellationToken
cancellationTokn)    {
        // Shutdown procedure
    }
}
```

Key responsibilities:

- Service initialization and shutdown
- Component lifecycle management
- Exception handling and recovery
- Background worker coordination

EventChannel

The `EventChannel` class is a central message bus for the entire system, handling all event routing between components. It's implemented as a singleton to ensure a single point of event processing.

csharp



```
public sealed class EventChannel
{
    private static readonly EventChannel instance = new
EventChannel();    private static EPRuntime esperRuntime;
```

```

        // Statistics tracking      public static long EventsPerSecond { get
{ return eventsPerSecond; } }      public static long TotalEvents { get
{ return totalEvents; } }

        // Provides access to the Esper runtime
public static EPRuntime EsperRuntime
{
get {
        // Initialize if needed
return esperRuntime;
}
}

        // Sends an event through the channel      public
static void Send(WintapMessage streamedEvent)
{
        // Process and route the event
}

        // Compiles and deploys an EPL statement      public static
EPDeployment compileDeploy(EPRuntime runtime, String epl)
{
        // Compile and deploy EPL code
}
}

```

The EventChannel provides:

- Event routing and distribution to subscribers
- Event buffering for PID resolution
- Performance statistics tracking
- Integration with the Esper EPL engine
- Query management for the Workbench interface

StateManager

The `StateManager` maintains the current state of the Wintap system, persisting configuration and providing status information to other components. Like `EventChannel`, it's a singleton. `csharp` 

```
public sealed class StateManager
{
    private static readonly StateManager state = new
    StateManager();

    // Global state properties
    public static
    Guid AgentId { get; private set; }    public
    static bool UserBusy { get; set; }    public
    static string ActiveUser { get; set; }    public
    static int PidFocus { get; set; }    public
    static bool OnBatteryPower { get; set; }

    // State access
    public static StateManager State
    {
        get { return
state; }
    }

    // State update methods
    internal static void
    SetWintapSettings(Dictionary<string, bool> settings)    {
        // Update settings
    }
}
```

Key state information includes:

- Agent identity and configuration
- User session status
- System power state
- Active process focus
- Collector enablement state
- Drive mappings (for Windows file path resolution)

Watchdog

The `Watchdog` class monitors system resource usage and Wintap component health, taking corrective action when necessary. `csharp` 

```
class Watchdog
{
    // Performance breach tracking
    internal static bool PerformanceBreach;

    // Monitors process runtime    protected internal
    void ProtectedRun(Runnable runnable)
    {
        // Execute with timeout monitoring
    }

    // Checks system resource usage    private void
    Watchdog_DoWork(object sender, DoWorkEventArgs e)
    {
        // Monitor CPU and memory usage
        // Take action if thresholds are exceeded
    }
}
```

The Watchdog provides:

- CPU and memory usage monitoring
- Plugin execution timeouts
- Graceful recovery from performance issues
- Logging of performance breaches
- Automatic restart capabilities

WintapLogger

The `WintapLogger` provides a centralized logging system for all Wintap components. It supports various log levels and output destinations.


```

public
{

    sealed class WintapLogger

    private static readonly WintapLogger _instance = new WintapLogger();
    private readonly ComponentLogger _defaultLogger;
    public static WintapLogger Log =>
_instance;

    // Log with various metadata    public void Append(string entry, LogLevel
targetVerbosity,        bool logToEventLog = false, EventLogEntryType eventLogType =
EventLogEntryType.Informat        int eventId = 1000,
        [CallerMemberName] string memberName = "",
        [CallerFilePath] string sourceFilePath = "")
    {
        // Log to file and optionally to Windows Event Log
    }
}

```

The logging system includes:

- Multiple verbosity levels (Debug, Info, Warn, Error, Fatal)
- File-based logging with rotation
- Windows Event Log integration
- Caller context capture Thread-
- safe operation

Collector Framework

The Collector Framework provides the foundation for gathering telemetry data from various sources. It defines base classes and interfaces that specific collectors implement to tap into system event sources.

```
public  
{
```

BaseCollector

The `BaseCollector` is the abstract base class for all collectors in the system. It provides common functionality and a consistent interface.

```
abstract class BaseCollector  
  
    internal string CollectorName { get; set; }  
  
    public virtual bool Start()  
    {  
return true;  
    }        public virtual  
void Stop()  
    {  
    }        internal  
BaseCollector()  
    {  
  
    }  
}
```

Key features:

- Common collector interface
- Naming and identification
- Start/stop lifecycle methods
- Statistics tracking infrastructure

```
public
{
```

BaseWinCollector

The `BaseWinCollector` extends `BaseCollector` with Windows-specific functionality, particularly for path normalization and ETW integration.

```
class BaseWinCollector : BaseCollector

internal WintapLogger log;
internal Properties.Settings
config;
internal bool enabled;

    public enum PathTypeEnum { Windows, WindowsShort, Relative, Unix, Win32File,
Win32Device, N    public enum CollectorTypeEnum { General, ETW }
    internal (string ProcessPath, string CommandLine) TranslateProcessPath(string
processName, {
        // Path normalization Logic
    }    internal string
TranslateFilePath(string filePath)
    {
        // File path normalization
    }
}
```

The `BaseWinCollector` provides:

- Windows path normalization
- Command line parsing
- Event parsing utilities
- Performance monitoring

```
public  
{
```

EtwProviderCollector

The `EtwProviderCollector` builds on BaseWinCollector to provide specialized functionality for ETW event collection.



```
csharp
internal abstract class EtwProviderCollector : BaseWinCollector
{
    public string EtwSessionName { get; set; }
    public BaseWinCollector.CollectorTypeEnum CollectorType;
    public string EtwProviderId { get; set; }    public
    TraceEventLevel EventLevel { get; set; }    public ulong
    TraceEventFlags { get; set; }
        public virtual void
    Process_Event(TraceEvent obj)
    {
        // Generic ETW event processing
    }    private void etwListenerThread_DoWork(object sender,
    DoWorkEventArgs e)
    {
        // ETW listening thread implementation
    }
}
```

Key features:

- ETW session management
- Event provider configuration
- Event filtering and processing
- Automatic reconnection

Collector Types

Wintap includes several collector implementations:

1. **ProcessCollector**: Gathers process creation, termination, and metadata events.
2. **FileCollector**: Tracks file system operations.
3. **TcpCollector** and **UdpCollector**: Monitor network activity.
4. **RegistryCollector**: Tracks Windows registry changes.
5. **ImageLoadCollector**: Monitors DLL and module loading.
6. **KernelAPICallCollector**: Tracks calls to sensitive kernel APIs.
7. **MicrosoftWindowsWin32kCollector**: Monitors user interface events.
8. **MemoryMapCollector**: Tracks memory allocations and usage.

Creating Custom Collectors

To create a custom collector:

1. Inherit from the appropriate base class:

csharp



```
internal class MyCustomCollector : EtwProviderCollector
{
    public MyCustomCollector() :
    base()
    {
        CollectorName = "MyCustom";
        EtwProviderId = "YOUR-PROVIDER-GUID";
    }
}
```

2. Implement the event processing logic:

csharp



```
public override void Process_Event(TraceEvent obj)
{
    base.Process_Event(obj);

    // Create a WintapMessage
    WintapMessage msg = new WintapMessage(obj.TimeStamp, obj.ProcessID,
        WintapMessage.MessageTypeEnum.GENERIC_MESSAGE);

    // Set additional properties

    // Send the event through the channel
    EventChannel.Send(msg);
}
```

3. Register the collector in the subscription manager or configuration.

Data Model

The Wintap data model centers around the WintapMessage class, which serves as a container for all telemetry data flowing through the system. This consistent structure enables uniform processing and querying of diverse event types.

WintapMessage

The `WintapMessage` class is the core data structure for all events in the system. It combines common metadata with type-specific details. `csharp` 

```
public class WintapMessage
{
    public enum MessageTypeEnum { PROCESS, FILE, REGISTRY, /* ...
    */ }
    public enum ActivityTypeEnum { Start, Stop, Read, Write, /*
    ... */ }

    public WintapMessage(DateTime eventTime, int processId, MessageTypeEnum
eventSourceName)
    {
        this.EventTime =
eventTime.ToFileTimeUtc();
        this.PID =
processId;
        this.MessageType = eventSourceName;
        this.ReceiveTime = DateTime.Now.ToFileTimeUtc();
    }

    // Common metadata
    public MessageTypeEnum
MessageType { get; set; }
    public long EventTime
{ get; set; }
    public long ReceiveTime { get;
set; }
    public int PID { get; set; }
    public
string PidHash { get; set; }
    public string
ProcessName { get; set; }
    public
ActivityTypeEnum ActivityType { get; set; }
    public string CorrelationId { get; set; }
    public
string ActivityId { get; set; }

    // Event-specific data containers
    public
ProcessObject Process { get; set; }
    public
TcpConnectionObject TcpConnection { get; set; }
    public
UdpPacketObject UdpPacket { get; set; }
    public
FileActivityObject File { get; set; }
    public
RegActivityObject Registry { get; set; }
    //
Additional event-specific objects...
}
```

Event Type Objects

Each event type has a corresponding object class that contains the specific data fields for that event type.

For example:



```

public
{
    class ProcessObject : WintapBase

        public int ParentPID { get; set; }
    public string ParentPidHash { get; set; }
    public string Name { get; set; }    public
    string Path { get; set; }    public string
    CommandLine { get; set; }    public string
    User { get; set; }    public string MD5 {
    get; set; }    public string SHA2 { get;
    set; }    // Additional properties...
}

public class FileActivityObject : WintapBase
{    public string Path { get; set; }
    public int BytesRequested { get; set; }
    // Additional properties...
}

```

All event-specific classes inherit from WintapBase , which provides common functionality: csharp



```

public abstract class WintapBase
{    public ExpandoObject
    ToDynamic()
    {
        // Convert to dynamic object for flexible serialization
    }
}

```

```
public
```

```
{
```

Message Type Enumeration

The `MessageTypeEnum` defines all supported event types:

```
enum MessageTypeEnum
```

```
PROCESS,  
PROCESS_PARTIAL,  
TCP_CONNECTION,  
UDP_PACKET,  
FILE,  
REGISTRY,  
IMAGE_LOAD,  
FOCUS_CHANGE,  
SESSION_CHANGE,  
WAIT_CURSOR,  
WMI,  
THREAD,  
GENERIC_MESSAGE,  
MICROSOFT_WINDOWS_CPU_TRIGGER,  
MICROSOFT_WINDOWS_GROUP_POLICY,  
MEMORY_MAP,  
KERNEL_API_CALL,  
EVENT_LOG_EVENT,  
SYSDIG,  
WINTAP_ALERT
```

```
}
```

Activity Type Enumeration

The `ActivityTypeEnum` defines all supported activity types, which specify the action being performed:

```
enum ActivityTypeEnum
```

```
public
{

    Start,
    Stop,
    Refresh,
    Rundown,
    Load,
    Unload,
    Read,
    Write,
    DeleteValue,
    CreateKey,
    DeleteKey,
    // Network-related activities
    TcpIpConnect,
    TcpIpSend,
    TcpIpRecv,
    UdpIpSend,
    UdpIpRecv,
    // Additional activity types...
    Other
}
```

Object Identity and Hashing

Wintap uses a hashing system to create consistent identifiers for objects, allowing for correlation across events. The IdGenerator class provides methods for generating these hashes:



```
csharp
internal class IdGenerator
{
    internal string GenKeyForProcess(string inContext, string
inHostName,          string inAgentId, int inPid, long inFirstEventTime,
string msgType)
    {
        // Generate a unique hash for a process
    }
    internal string GenKeyForFile(string inContext,
string inHostName,          string inAgentId, string inPath)
    {
        // Generate a unique hash for a file
    }

    // Additional hash generation
    methods... }

```

Working with the Data Model

When creating events:

1. Create a WintapMessage with appropriate type:

```
csharp
```

```
WintapMessage msg = new WintapMessage(DateTime.Now, processId,
WintapMessage.MessageTypeEnum.PROCESS);
```



2. Set the activity type:

```
csharp
```

```
msg.ActivityType = WintapMessage.ActivityTypeEnum.Start;
```



3. Create and populate the event-specific object:

```
csharp
```

```
msg.Process = new
WintapMessage.ProcessObject(); msg.Process.Name
= "example.exe"; msg.Process.Path =
@"c:\example\example.exe";
msg.Process.ParentPID = parentProcessId;
```



4. Send the message through the EventChannel:

csharp

```
EventChannel.Send(msg);
```

ETL Pipeline

The Extract, Transform, and Load (ETL) pipeline processes raw telemetry data into structured, storable formats. This pipeline provides data enrichment, normalization, and persistence.

Extract Layer

The extraction layer consists of `Sensor` classes that consume events from the EventChannel and

prepare them for transformation. csharp 

```
internal abstract class Sensor
{
    protected Sensor(string
query)
    {
        // Initialize and register the EPL query
    }
    protected virtual void HandleSensorEvent(EventBean
sensorEvent) {
        // Process the event and prepare for transformation
    }
    internal void
Save(ExpandoObject obj)
    {
        // Queue the object for serialization
    }
}
```

Specialized sensors exist for each event type:

- `PROCESS_SENSOR` : Handles process events
- `FILE_SENSOR` : Handles file system events
- `TCPCONNECTION_SENSOR` : Handles TCP network events
- `UDPPACKET_SENSOR` : Handles UDP network events
- `REGISTRY_SENSOR` : Handles registry events



Each sensor implements its own `HandleSensorEvent` method to process events from EventChannel according to its specific requirements.

Transform Layer

The transformation layer, primarily implemented in the `Transformer` class, enriches and normalizes data before storage. `csharp`

```
internal class Transformer
{
    internal static string context =
        "11n1";

    internal static ProcessConnIncrData CreateProcessConn(EventBean
newEvent,          string _pidhash, List<NIC> activeNics)    {
        // Create and populate connection data
    }

    internal static IPv4Addr createIpAddr(string ip, uint
ipLong, string gw)    {
        // Create IP address data structure
    }

    // Additional transformation
    methods }
```

Key transformation operations include:

- Converting raw data to structured objects
- Generating unique identifiers and hashes
- Normalizing paths and addresses
- Enriching data with additional context
- Converting between data formats and types

Load Layer

The load layer persists transformed data to disk and prepares it for upload to external systems.

CacheManager

The `CacheManager` coordinates the caching and persistence of data: `csharp` 

```
internal class CacheManager
{
    internal ETLConfig etlConfig;    internal
    static ConcurrentQueue<dynamic> SendQueue;
        internal CacheManager(ETLConfig
_config)
    {
        // Initialize cache management
    }    internal
void Start()
{
    // Begin processing queue
}    internal
void Stop()
{
    // Flush remaining data and shut down
}    private
void doMerge()
{
    // Merge data for more efficient storage
}    private
void upload()
{
    // Upload data to configured destinations
}
}
```

ParquetWriter

The `ParquetWriter` handles the serialization of data to the Apache Parquet format:

```
internal class ParquetWriter : FileWriter
{
    internal int Backlog { get { return batches.Count;
} }

    internal void Add(Batch
batch)    {
        // Queue batch for writing
    }
```



```
csharp
    }
    internal async Task<string>
Write(Batch.SensorData dataSet)
    {
        // Write data to Parquet file
    }
}
```

Upload Adapters

Upload adapters implement the `IUpload` interface to send data to various destinations: csharp 

```
public interface IUpload
{
    event EventHandler<string>
UploadCompleted;
    string Name { get; set; }
}

bool PreUpload(Dictionary<string,string> parameters);
Task<bool> Upload(string localFile, Dictionary<string, string> parameters);
bool PostUpload();
}
```

Implementations include:

- `SMBFileShareAdapter` : Uploads to network file shares
- `InstanceProfileAdapter` : Uploads to AWS S3 using instance profiles

Configuring the ETL Pipeline

The ETL pipeline is configured via the `ETLConfig` class:

```
public class ETLConfig
{
    public string LogLevel { get; set; }
    public string SensorProfile { get; set; }
    public int SerializationIntervalSec { get; set; }
    public int UploadIntervalSec { get; set; }
    public bool WriteToParquet { get; set; }
    public bool WriteToCsv { get; set; }
    public List<Adapter> Adapters = new List<Adapter>();
}

public class
Adapter
```




```
csharp
{
    public string Name { get; set; }
    public bool Enabled { get; set; }
    public Dictionary<string, string> Properties;
}
}
```

This configuration can be modified via a JSON configuration file or programmatically.

Plugin System

Wintap's plugin system enables extensibility without modifying core code. Plugins are loaded into isolated domains for improved stability and security.

Plugin Architecture

The plugin system consists of several key components:

1. **PluginManager**: Coordinates plugin discovery, loading, and lifecycle management
2. **PluginDomainManager**: Handles the creation and management of isolated AppDomains for plugins
3. **IsolatedPluginCatalog**: A custom MEF catalog that loads plugins into isolated domains
4. **PluginExceptionHandler**: Provides global exception handling for plugin execution

Plugin Interfaces

Plugins implement one or more of these interfaces:

```
// For subscribing to specific event types
```

```
public interface ISubscribe
{
    void Subscribe(WintapMessage
eventMsg);    EventFlags Startup();
    void Shutdown(); }
```

```
// For subscribing to raw ETW events
```

```
public interface ISubscribeEtw
{
    void Subscribe(WintapMessage
wintapMessage);    List<string> Startup();
    void Shutdown(); }
```



```
csharp
// For scheduled task execution
public interface IRun
{
    void Run();
    RunManifest RunStartup();
    void RunShutdown(); }

// For custom EPL queries
public interface IQuery
{
    List<EventQuery> Startup();
    void Process(QueryResult result);
    void Shutdown(); }

// For providing custom events
public interface IProvide
{
    void Startup();    event
    EventHandler<ProviderEventArgs> Events;
    void RaiseEvent();    void Shutdown();
}
```

Each interface has a corresponding metadata interface (e.g., `ISubscribedData`) that provides information about the plugin.

Plugin Loading Process

1. The PluginManager scans the plugin directory for subdirectories
2. Each subdirectory is treated as a potential plugin container
3. Assemblies are examined for MEF export attributes
4. Valid plugins are loaded into isolated AppDomains
5. Plugin Startup methods are called
6. The system registers event handlers based on plugin requirements

csharp



```
public class PluginManager
{
    internal void RegisterPlugins(Watchdog
_watchdog)
    {
        LoadPluginAssemblies();
        RegisterEventHandlers();
        StartPluginScheduler();
    }
    private void
LoadPluginAssemblies()
    {
        isolatedCatalog = new
IsolatedPluginCatalog(Strings.FilePluginPath);
        CompositionContainer(isolatedCatalog);
        mefContainer.ComposeParts(this);
    }
}
```

Plugin Isolation

Plugins run in isolated domains to prevent them from affecting the stability of the core system:

csharp



```
public class PluginDomain : IDisposable
{
    private readonly AssemblyLoadContext
_loadContext;
    private readonly string _pluginId;
```



```
private readonly string _pluginPath;    private
Assembly _pluginAssembly;
    public void
LoadPlugin()
{
    _pluginAssembly = _loadContext.LoadFromAssemblyPath(_pluginPath);
}    public
void Unload()
{
    _loadContext.Unload();
}
}
```

Creating Plugins

To create a plugin:

1. Create a class library project targeting .NET 6.0+
2. Reference the Wintap.API assembly
3. Implement one or more plugin interfaces
4. Add MEF export attributes to your classes
5. Place the compiled assembly in a subdirectory of the plugins folder

Example plugin implementation:

csharp

```
[Export(typeof(ISubscribe))]
[ExportMetadata("Name", "MyCustomPlugin")]
public class MyCustomPlugin : ISubscribe
{    public void Subscribe(WintapMessage
eventMsg)
{
    // Process events here
}    public
EventFlags Startup()
{
    // Return the event types to subscribe to
return EventFlags.Process | EventFlags.FileActivity;
}    public void
Shutdown()
```

```

    {
        // Cleanup resources
    }
}

```

Plugin Configuration

Plugins can store configuration in the Windows registry: `csharp`



```

using (var pluginKey = Registry.LocalMachine.CreateSubKey(
    Strings.RegistryPluginPath + "\\\" + pluginName,
    RegistryKeyPermissionCheck.ReadWriteSubTree))
{
    pluginKey.SetValue("ConfigSetting",
        "Value");    pluginKey.Flush();
}

```

Web API and User Interface

Wintap provides a web-based interface for querying data and managing the system. This interface is built on ASP.NET Core and Angular.

API Controllers

The REST API is implemented as a set of ASP.NET Core controllers: `csharp`



```

[ApiController]
[Route("api/[controller]")] public class
StreamsController : ControllerBase
{
    [HttpPost]
    [Route("api/streams")]    public IActionResult
Post([FromBody] EsperQuery q)
    {
        // Handle query management
    }

    [HttpGet]
    [Route("api/Streams")]    public
IActionResult GetAllStatements()

```



```
{  
    // Return all saved queries  
}  
  
// Additional API methods...  
}
```

Key controllers include:

- `StreamsController` : Manages EPL queries
- `EsperServiceController` : Provides status and metrics
- `TreeController` : Exposes process tree data
- `LLMController` : Interfaces with the AI assistant
- `WintapServiceController` : Manages service settings

SignalR Hubs

Wintap uses SignalR for real-time communication between the server and web UI:

csharp

```
public class WorkbenchHub : Hub  
{  
    public async Task Send(string  
queryResult)  
    {  
        await Clients.All.SendAsync("ReceiveMessage",  
queryResult);  
    }  
}  
  
public class InferenceHub : Hub  
{  
    public async Task Send(Inference  
inference)  
    {  
        await Clients.All.SendAsync("ReceiveMessage",  
inference);  
    }  
}
```

These hubs enable:

- Real-time query results
- AI assistant responses
- Process tree updates
- ETW event streaming

Angular Frontend

The Web UI is built with Angular and PrimeNG components: typescript



```
@Component({  selector: 'app-querybuilder',
  templateUrl: './querybuilder.component.html',
  styleUrls: ['./querybuilder.component.scss']
}) export class QuerybuilderComponent implements AfterViewInit, OnInit,
OnDestroy {  // Component implementation
}
```

Key UI components:

- `QueryBuilderComponent` : Interface for creating and managing queries
- `TreeviewComponent` : Visualizes process hierarchies
- `EtwExplorerComponent` : Explores available ETW providers
- `ChatComponent` : Interfaces with the AI assistant

API Integration

The Angular frontend communicates with the Wintap API using HTTP requests and SignalR:

typescript



```
private initSignalRConnection() {
  this.connection = new HubConnectionBuilder()
    .withUrl('/signalr/workbenchHub')
    .withAutomaticReconnect([0, 2000, 10000, 30000])
    .build();

  this.connection.start()    .then(() => {
    console.log('SignalR connection established');
  });
}
```



```
    })
    .catch(error => { console.error('SignalR connection error:', error);
this.showToast('error', 'Connection Error', 'Failed to connect to real-time updates');
});

this.connection.on('ReceiveMessage', (message: EsperResult, status: string) =>
{ // Process real-time messages
});
}
```

Query Language

Wintap uses Esper Event Processing Language (EPL) for real-time event querying and analysis. EPL is a SQL-like language specifically designed for processing streams of events.

EPL Basics

EPL queries follow this general structure:

```
sql

SELECT [properties]
FROM [event_stream] [optional window]
WHERE [conditions]
[additional clauses]
```

Example: sql 

```
SELECT * FROM WintapMessage
WHERE MessageType = "PROCESS" AND ActivityType = "Start"
```

Common Query Patterns

1. Basic Filtering

sql

```
SELECT * FROM WintapMessage
WHERE MessageType = "PROCESS" AND Process.Name LIKE "%chrome%"
```



2. Time Windows

sql



```
SELECT * FROM WintapMessage.win:time(30 sec)
WHERE MessageType = "FILE" AND File.Path LIKE "%temp%"
```

3. Event Patterns

sql



```
SELECT * FROM PATTERN [ every
p=WintapMessage(MessageType="PROCESS", ActivityType="Start") ->
f=WintapMessage(MessageType="FILE", PID=p.PID)
WHERE timer:within(5 sec)
]
```

4. Aggregation

sql



```
SELECT ProcessName, count(*) as EventCount
FROM WintapMessage.win:time(5 min)
WHERE MessageType = "TCP_CONNECTION"
GROUP BY ProcessName
ORDER BY EventCount DESC
```

Integration with EventChannel

EPL queries are compiled and deployed through the EventChannel: `csharp`

```
EPDeployment deployment = EventChannel.compileDeploy(EventChannel.EsperRuntime, eplQuery);
deployment.Statements[0].Events += (sender, e) =>
{
    // Process query results
};
```



Workbench Query Builder

The Workbench UI provides a query builder with:

- Syntax highlighting
- Autocomplete suggestions
- Query templates
- Real-time result visualization
- Query management (save, load, modify)

Security Considerations

Security is a critical aspect of Wintap, as it handles sensitive system information and requires elevated privileges for certain operations.

Service Security Context

Wintap runs as a Windows service with the following security context:

- Service account: LocalSystem by default
- Required privileges: SeDebugPrivilege for process access

To run with reduced privileges, modify the service configuration to use a custom service account with the minimum required permissions.

Data Security

Telemetry data is sensitive and should be protected:

- Local data is stored in the Parquet format in `%ProgramData%\Wintap\`
- Use NTFS permissions to restrict access to this directory
- Implement encryption for data at rest if required

For data in transit:

- Use HTTPS for web API communications
- Implement authentication for API access
- Consider TLS for upload adapters

Plugin Security

Plugins represent a potential security risk:

- Plugins run in isolated AppDomains but still within the Wintap process
- Only load signed plugins in production environments
- Implement plugin activity monitoring through the Watchdog
- Use the PluginExceptionHandler to prevent plugin failures from affecting the core system

ETW Security

ETW provides deep system visibility and requires careful handling:

- Limit ETW provider subscriptions to what is necessary
- Be aware that some ETW data may contain sensitive information
- Follow the principle of least privilege when collecting data

API Security

The web API should be secured:

- Implement authentication for API access
- Use HTTPS for all communications
- Consider implementing rate limiting
- Validate all input parameters

Deployment and Configuration

Proper deployment and configuration ensures Wintap operates efficiently and securely in your environment.

Installation

Wintap is deployed as a Windows service:

1. Copy the Wintap binaries to the installation directory (default: `%ProgramFiles%\Wintap\`)
2. Install the service:

```
sc create Wintap binPath= "%ProgramFiles%\Wintap\Wintap.exe" start= auto
```

3. Set appropriate permissions on the installation directory
4. Start the service:

```
sc start Wintap
```

Configuration Files

Wintap uses several configuration files:

1. **Wintap.dll.config**: Main application configuration

xml

```
<configuration>
  <appSettings>
    <add key="LogLevel" value="Normal" />
    <add key="ApiPort" value="8099" />
    <!-- Additional settings -->
  </appSettings>
```

```
</configuration>
```

2. **ETLConfig.json**: ETL pipeline configuration

json



```
{
  "LogLevel": "Info",
  "SensorProfile": "Production",
  "SerializationIntervalSec": 30,
  "UploadIntervalSec": 300,
  "WriteToParquet": true,
  "WriteToCsv": false,
  "Adapters": [
    {
      "Name": "SMBFileShareAdapter",
      "Enabled": true,
      "Properties": {
        "UNCPath": "\\server\\share"
      }
    }
  ]
}
```

Registry Configuration

Wintap stores settings in the registry under `HKLM\SOFTWARE\Wintap\` :

- `AgentId` : Unique identifier for this Wintap instance
- Collector enablement settings
- Plugin configurations
- Session state information

Performance Profiles

Wintap supports different performance profiles:

1. **Production**: Balanced performance with throttling

- MaxCPU: 20%
- MaxMem: 700MB
- Event throttling: Enabled

2. **Developer:** Higher resource limits for testing MaxCPU: Unlimited

- MaxMem: 950MB
- Event throttling: Disabled

3. **Minimal:** Reduced footprint for resource-constrained systems

- MaxCPU: 10%
- MaxMem: 350MB
- Limited collector set

Configure the profile in Settings.Default.Profile.

Collector Configuration

Enable or disable collectors via the configuration: `csharp`



```
Properties.Settings.Default.ProcessCollector = true;
Properties.Settings.Default.FileCollector = true;
Properties.Settings.Default.TcpCollector = false;
// Additional collector settings...
Properties.Settings.Default.Save();
```

Or via the WintapServiceController API: `csharp`



```
Dictionary<string, bool> settings = new Dictionary<string,
bool>(); settings["Process"] = true; settings["File"] = true;
settings["Tcp"] = false;
// Additional settings...
StateManager.SetWintapSettings(settings);
```

Troubleshooting and Diagnostics

Effective troubleshooting is essential for maintaining a healthy Wintap deployment. This section covers common issues, diagnostic tools, and resolution strategies.

Logging System

Wintap uses a multi-level logging system:

csharp



```
public enum LogLevel
{
    Always = -1, // Legacy compatibility
    Trace = 0,
    Debug = 1,
    Info = 2,
    Warn = 3,
    Error = 4,
    Fatal = 5
}
```

To access logs:

- File logs: `%ProgramData%\Wintap\Logs\Wintap.log`
- Windows Event Logs: Application log, source "Wintap" Configure log verbosity: csharp



```
Properties.Settings.Default.LoggingLevel = "Debug";
Properties.Settings.Default.Save();
```

Common Issues and Solutions

High CPU Usage

Symptoms: System performance degradation, Watchdog restarts

Diagnostics:

1. Check logs for "WARN problem in Watchdog" entries
2. Monitor CPU usage with Performance Monitor

3. Review enabled collectors

Solutions:

1. Disable high-volume collectors:

csharp



```
Properties.Settings.Default.FileCollector = false;  
Properties.Settings.Default.Save();
```

2. Increase throttling thresholds:

csharp



```
WintapProfile.MaxCPU = 30; // Increase CPU Limit to 30%
```

3. Filter ETW events more aggressively

Memory Leaks

Symptoms: Increasing memory usage over time, Watchdog restarts

Diagnostics:

1. Check logs for "ERROR in CacheManager" entries
2. Monitor Private Bytes in Performance Monitor
3. Review ETL pipeline configuration

Solutions:

1. Decrease serialization interval:

json



```
{  
  "SerializationIntervalSec": 15  
}
```

2. Implement more aggressive cache pruning:

csharp



```
long maxCacheSizeBytes = 128000000; // Reduce to 128MB
```


3. Check for plugin memory leaks and unload problematic plugins

Event Loss

Symptoms: Missing events, gaps in telemetry

Diagnostics:

1. Check logs for "DroppedEventCount" entries
2. Monitor ETW session statistics
3. Review EventChannel buffer size

Solutions:

1. Increase ETW buffer size:

csharp



```
EtwSession.BufferSizeMB = 500;
```

2. Reduce collector scope:

csharp



```
Properties.Settings.Default.CollectFileRead = false;
```

3. Implement event filtering closer to the source

Workbench Connectivity Issues

Symptoms: Web UI cannot connect, SignalR timeouts

Diagnostics:

1. Check logs for API errors
2. Verify port availability
3. Test local connectivity

Solutions:

1. Modify API port:

csharp



```
Properties.Settings.Default.ApiPort = 8100;
```

2. Ensure firewall rules allow the configured port

3. Check for conflicting web servers or services

Diagnostic Tools

1. **Wintap Workbench:** Use the Query Builder to examine real-time events
2. **Event Viewer:** Windows Event Logs contain Wintap service messages
3. **Performance Monitor:** Create a custom view with Wintap counters: Process% Processor Time for
 - Wintap.exe
 - Process\Private Bytes for Wintap.exe
 - .NET CLR Memory# Gen 0 Collections
4. **ETW Inspector:** Use the ETW Explorer in the Workbench to examine ETW providers

Performance Tuning

1. **Selective Collection:** Enable only necessary collectors
2. **Event Filtering:** Use targeted ETW provider flags
3. **Batch Processing:** Adjust serialization intervals
4. **Memory Management:** Configure appropriate cache sizes
5. **Query Optimization:** Refine EPL queries for efficiency

AI Assistant Integration

Wintap 7 introduces an integrated AI assistant that provides intelligent analysis of telemetry data and helps users understand system behavior.

Architecture

The AI integration consists of several components:

1. **LLMController:** ASP.NET Core controller that handles AI requests
2. **InferenceHub:** SignalR hub for real-time AI communication
3. **Semantic Memory:** Vector database for storing contextual information
4. **Chat Interface:** Web UI component for interacting with the AI

Semantic Kernel Integration

Wintap uses Microsoft's Semantic Kernel framework to interact with the AI models:

csharp

```
var aiBuilder = Kernel.CreateBuilder();

// Add the Ollama chat completion
service
aiBuilder.AddOllamaChatCompletion(
modelId: "gemma3:1b",      endpoint: new
Uri(OllamaEndpoint) );

// Add the Ollama embedding service
aiBuilder.AddOllamaTextEmbeddingGeneratio
n(      modelId: EmbeddingModel,
endpoint: new Uri(OllamaEndpoint) );

// Build the kernel with both services
var kernel = aiBuilder.Build();
```

RAG (Retrieval-Augmented Generation)

The AI assistant uses RAG to provide context-aware responses: csharp



```
// Create memory using the embedding service and SQLite
var sqliteMemoryStore =
SqliteMemoryStore.ConnectAsync(DatabasePath).GetAwaiter().GetResult();

ISemanticTextMemory memory = new MemoryBuilder()
    .WithLoggerFactory(kernel.LoggerFactory)
    .WithMemoryStore(sqliteMemoryStore)
    .WithTextEmbeddingGeneration(embeddingService)
    .Build();
```

This allows the assistant to:

1. Store Wintap documentation and knowledge
2. Retrieve relevant context for queries
3. Provide accurate, contextual responses



AI Request Handling

The LLMController processes inference requests: csharp



```
[HttpPut("Inference")] public async Task Put([FromBody]
PromptModel promptModel)
{
    // Retrieve relevant information from the semantic
    memory    StringBuilder builder = new StringBuilder();
    double minRel = Settings.Default.MinRelevance;
    await foreach (MemoryQueryResult result in
memory.SearchAsync(        collectionName, question, 3, minRel,
withEmbeddings: true))
    {
        builder.AppendLine(result.Metadata.Text);
    }

    // Add context to the chat
    if (builder.Length != 0)
    {
        chat.AddUserMessage("USER QUERY: " + question +
"\n\n" +
        "Retrieved data: " + builder.ToString());
    }

    // Get streaming response    await foreach
(StreamingChatMessageContent message in
ai.GetStreamingChatMessageContentsAsync(chat, settings))
    {
        Inference inf = new Inference() {
            Prompt = question,
            Response = message.Content,
            TokensUsed = 0
        };
        this.hubContext.Clients.All.SendAsync("ReceiveMessage", inf, "OK");    await
    }
}
```

Document Embedding

Users can upload documentation to enhance the AI's knowledge:

csharp

```

[HttpPost("Upload")] public async Task<IActionResult>
Upload(IFormFile file)
{
    using (var stream = new
MemoryStream())
    {
        await file.CopyToAsync(stream);
stream.Position = 0;        using (var reader =
new StreamReader(stream))
        {
            string fileContent = await reader.ReadToEndAsync();
            List<string> lines = TextChunker.SplitPlainTextLines(fileContent,
128);
            List<string> paragraphs =
TextChunker.SplitPlainTextParagraphs(
lines, chunkSize,
overlapSize, " ");
            for (int i = 0; i <
paragraphs.Count; i++)
            {
                if
(!string.IsNullOrEmpty(paragraphs[i]))
                {
                    await
memory.SaveInformationAsync(
collectionName,
paragraphs[i], $"paragraph{i}");
                }
            }
        }
    }
    return Ok();
}

```

Chat Interface

The web UI provides a chat interface for interacting with the AI assistant: typescript



```

@Component({
    selector: 'app-chat',
    templateUrl: './chat.component.html',
    styleUrls: ['./chat.component.scss']
}) export class ChatComponent implements OnInit {
    // Component implementation for AI chat interface
}

```

Working with the AI Assistant

The AI assistant can help with:



1. **Telemetry Analysis:** Examining patterns in collected data
2. **Query Generation:** Creating EPL queries for specific scenarios
3. **Troubleshooting:** Identifying potential issues from telemetry
4. **Documentation:** Explaining Wintap concepts and features Example query for the AI assistant:



Analyze the recent network connections from Chrome and identify any potentially suspicious connections."

"What processes have been accessing the registry keys under HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run?"

"Explain the memory usage pattern of svchost.exe over the past hour and suggest optimization strategies."

"Create an EPL query to detect potential data exfiltration via DNS requests."

Multi-Platform Support

Wintap 7 introduces a foundation for multi-platform support, enabling telemetry collection across Windows, Linux, and macOS environments.

Platform Abstraction Layer

The platform abstraction is implemented through a hierarchy of base classes:

BaseCollector (abstract)

- └─ BaseWinCollector (Windows-specific)
- └─ [Future] BaseUnixCollector (Linux/macOS)

The ``SubscriptionManager`` class handles platform detection and initialization of the appropriate collectors:

```
```csharp
public class
SubscriptionManager
{
 internal void
Start()
 {
 if
(RuntimeInformation.IsOSPlatform(OSPlatform.Windows))
```

```

 {
 winCollectors =
winSubMgr.Start();
 }
 else if
(RuntimeInformation.IsOSPlatform(OSPlatform.Linux))
 {
 linuxCollectors =
linuxSubMgr.Start();
 }
 else if
(RuntimeInformation.IsOSPlatform(OSPlatform.OSX))
 {
 // Mac support future implementation
 }
else
 {
 WintapLogger.Log.Append("Running on an unsupported platform", LogLevel.Info);
 }
 }
}

```

## Windows Implementation

Windows telemetry collection is primarily based on ETW (Event Tracing for Windows):

```

internal class WindowsSubscriptionManager
{
 internal List<BaseWinCollector>
Start()
 {
 List<BaseWinCollector> baseCollectors = new
List<BaseWinCollector>();
 // Start process collector first for process
attribution ProcessCollector pc = new
ProcessCollector(); pc.Start();
baseCollectors.Add(pc);

 // Start other Windows-specific
collectors // ... return
baseCollectors;
 }
}

```

ETW provides a rich set of event providers that expose system activity at various levels, from kernel operations to application behavior. The Windows implementation leverages these providers to collect comprehensive telemetry data with minimal overhead.

## Linux Implementation

Linux support is implemented through the LinuxSubscriptionManager :

```

public class LinuxSubscriptionManager
{
 internal List<BaseCollector>
Start()
 {

```



```
 List<BaseCollector> baseCollectors = new
List<BaseCollector>();
 // Start process collector
 ProcessCollector pc = new ProcessCollector();
pc.Start();

 // Start Sysdig collector for system calls
 SysdigCollector sysdig = new SysdigCollector(Path.Combine(
 Environment.GetFolderPath(Environment.SpecialFolder.CommonApplicationData),
 "Wintap", "Sysdig", "ygm-class-long-99.json"));
sysdig.Start();

baseCollectors.Add(pc);
baseCollectors.Add(sysdig);
 return
baseCollectors;
 }
}
```

The current Linux implementation uses Sysdig to capture system call data, which provides visibility into process, file, and network activity. This approach allows for detailed system monitoring without requiring kernel modifications or custom modules.

## macOS Implementation Strategy

For future macOS support, a similar approach would be used:

```
public class MacOSSubscriptionManager
{
 internal List<BaseCollector>
Start()
```

```
{
 List<BaseCollector> baseCollectors = new List<BaseCollector>();

 // Implement macOS-specific collectors
 // Potentially Leverage Endpoint Security Framework, DTrace, or kauth
 return
baseCollectors;
}
```

The macOS implementation would leverage platform-specific APIs and technologies:

1. **Endpoint Security Framework:** Apple's modern API for security events
2. **DTrace:** Comprehensive system tracing capabilities
3. **kauth:** Kernel authorization callbacks for monitoring system operations
4. **OpenBSM:** Audit framework for system event recording

## Creating Cross-Platform Collectors

To create a truly cross-platform collector, developers should:

1. Inherit from the base `BaseCollector` class:



csharp

```
public class CrossPlatformCollector : BaseCollector
{
 public override bool
 Start()
 {
 if
 (RuntimeInformation.IsOSPlatform(OSPlatform.Windows))
 {
 // Windows-specific initialization
 }
 else if
 (RuntimeInformation.IsOSPlatform(OSPlatform.Linux))
 {
 // Linux-specific initialization
 }
 else if
 (RuntimeInformation.IsOSPlatform(OSPlatform.OSX))
 {
 // macOS-specific initialization
 }
 return true;
 }
}
```

2. Use platform detection to initialize appropriate data sources
3. Normalize platform-specific data into the common WintapMessage format
4. Handle platform-specific paths, identifiers, and error conditions
5. Implement appropriate cleanup in the Stop method

## Platform-Specific Considerations

### Windows Considerations

- ETW provides low-level access but requires elevated privileges
- Some ETW providers are only available on specific Windows versions
- Registry operations are Windows-specific and need alternatives on other platforms
- Windows paths use backslashes and drive letters

### Linux Considerations

- Requires appropriate capabilities or root access for system monitoring
-

Process information is available via procfs (/proc)

- File system events can be monitored via inotify/fanotify
- Network monitoring typically uses netlink sockets or packet capture
- Audit framework provides security-relevant events

## macOS Considerations

- System Integrity Protection (SIP) restricts system monitoring
- Endpoint Security Framework requires explicit entitlements
- DTrace is available but restricted on recent versions
- Process and file monitoring APIs differ from both Windows and Linux

## Strategies for Linux Data Collection

Several options exist for collecting telemetry data on Linux systems:



### 1. **Sysdig**: Currently implemented for system call tracing

csharp

```
internal class SysdigCollector : BaseCollector
{
 internal SysdigCollector(string
jsonFile)
 {
 jsonInfo = new
FileInfo(jsonFile);
 }
 internal
void Start()
 {
 // Process JSON data from sysdig
 // Convert to WintapMessage format
 // Send events through EventChannel
 }
}
```

### 2. **eBPF (Extended Berkeley Packet Filter)**:

- Efficient kernel-level tracing
- Low overhead monitoring of system calls, network traffic, etc.
- Requires a wrapper or helper program to interface with C#

### 3. **Linux Audit Framework:** Security-focused event collection

- Configurable rules for event generation
- Available on most Linux distributions

### 4. **procfs/sysfs:**

- Direct access to process and system information
- No special permissions required for basic information
- Limited real-time notification capabilities

### 5. **Inotify/Fanotify:**

- File system event monitoring
- Notifications for file creation, modification, etc.
- Accessible through .NET APIs

## Strategies for macOS Data Collection

Future macOS support could leverage these technologies:

### 1. **Endpoint Security Framework:**

- Modern API specifically for security monitoring
- Comprehensive event types (process, file, network)
- Requires Apple Developer Program membership and entitlements

### 2. **DTrace:**

- Dynamic tracing framework
- Flexible instrumentation capabilities
- May require System Integrity Protection adjustments

### 3. **OpenBSM:**

- Audit framework for system event recording
- Compatible with Linux auditd
- Requires appropriate permissions

## Unified Data Model

Despite platform differences, all collectors output data in the common `WindowsMessage` format:

csharp



```
WintapMessage msg = new WintapMessage(DateTime.Now,
processId, WintapMessage.MessageTypeEnum.PROCESS);
msg.ActivityType = WintapMessage.ActivityTypeEnum.Start;
msg.Process = new WintapMessage.ProcessObject();
// Set platform-specific data
EventChannel.Send(msg);
```

This ensures consistent handling throughout the ETL pipeline and enables cross-platform analysis.

## API Reference

This section provides detailed reference information for key Wintap APIs.

### EventChannel API

csharp



```
// Send an event through the system public static
void Send(WintapMessage streamedEvent);

// Compile and deploy an EPL query public static EPDeployment
compileDeploy(EPRuntime runtime, String epl);

// Manage a workbench query internal static EsperQuery
ManageWorkbenchQuery(EsperQuery q);

// Get all saved workbench queries
internal static List<EsperQuery> getWorkbenchState();
```

### StateManager API

```
// Access static state properties public static
Guid AgentId { get; private set; } public static
bool UserBusy { get; set; } public static string
ActiveUser { get; set; } public static DateTime
LastUserActivity { get; set; } public static bool
OnBatteryPower { get; set; } public static int
PidFocus { get; set; }
```

csharp



```
// Update Wintap collector settings internal static void
SetWintapSettings(Dictionary<string, bool> settings);

// Get the last boot time internal static
DateTime refreshLastBoot();

// Get the local IP address
public static string GetLocalIpAddress();
```

## WintapLogger API

csharp



```
// Log a message with various metadata public void Append(string entry, LogLevel
targetVerbosity, bool logToEventLog = false, EventLogEntryType eventLogType =
EventLogEntryType.Information, int eventId = 1000,
 [CallerMemberName] string memberName = "",
 [CallerFilePath] string sourceFilePath = "");

// Close the log
public void Close();

// Access the logger instance
public static WintapLogger Log => _instance;
```

## Plugin API Interfaces

```
// Subscribe to Wintap events
public interface ISubscribe
{
 void Subscribe(WintapMessage
eventMsg); EventFlags Startup();
void Shutdown(); }

// Execute scheduled tasks
public interface IRun
{
 void Run();
RunManifest RunStartup();
void RunShutdown(); }
```

```
// Process query results
public interface IQuery
{
 List<EventQuery> Startup();
 void Process(QueryResult result);
 void Shutdown();
}
```

## REST API Endpoints

### Streams API

- `GET /api/streams` : Get all queries
- `GET /api/streams/{name}` : Get a specific query
- `POST /api/streams` : Create or update a query
- `DELETE /api/streams` : Delete all queries

### Esper Service API

- `GET /api/EsperService` : Get performance metrics

### Tree API

- `GET /api/Tree` : Get process tree data



## LLM API

- `PUT /api/LLM/Inference` : Submit a query to the AI assistant
- `POST /api/LLM/Clear` : Clear the conversation history
- `POST /api/LLM/Upload` : Upload a document to the knowledge base

## Wintap Service API

- `GET /api/WintapService` : Get current configuration
- `POST /api/WintapService` : Update configuration

## Contributing Guidelines

Contributions to Wintap are welcome. This section outlines the process and standards for contributing.

### Code Style

Wintap follows these coding standards:

- Use Microsoft's C# Coding Conventions
- Follow the .NET Framework Design Guidelines
- Use meaningful variable and method names
- Include XML documentation comments for public APIs
- Keep methods focused and concise
- Use nullable reference types appropriately
- Add appropriate exception handling

## Development Environment Setup

1. Install Visual Studio
2. Install the .NET SDK
3. Install the Angular CLI for web UI development
4. Clone the Wintap repository
5. Open the solution in Visual Studio
6. Build the solution



## Testing Requirements

All contributions should include appropriate tests:

1. **Unit Tests:** Tests for individual components
2. **Integration Tests:** Tests for component interactions
3. **Performance Tests:** For performance-sensitive changes
4. **UI Tests:** For web interface changes

## Pull Request Process

1. Fork the repository
2. Create a branch for your feature
3. Make your changes
4. Add or update tests
5. Ensure all tests pass
6. Update documentation
7. Submit a pull request

## Code Review Criteria

Pull requests are evaluated based on:

- Adherence to coding standards
- Test coverage
- Performance impact
- Security implications
- Documentation quality
- Compatibility with existing code

## Licensing Information

### MIT License

Wintap is licensed under the MIT License:

MIT License

Copyright (c) 2025, Lawrence Livermore National Security, LLC.

Produced at the Lawrence Livermore National Laboratory.  
All rights reserved.

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

## LLNL Release Information

LLNL Release Number: LLNL-CODE-7845213

This work was produced under the auspices of the U.S. Department of Energy by Lawrence Livermore National Laboratory under Contract DE-AC52-07NA27344.

## Third-Party Licenses

Wintap incorporates several third-party libraries, each with its own license:

1. **Esper**: EPL complex event processing engine (GPL v2 with classpath exception)
2. **Apache Parquet**: Columnar storage format (Apache License 2.0)
3. **Microsoft.SemanticKernel**: AI integration framework (MIT License)
4. **Angular**: Web application framework (MIT License)
5. **PrimeNG**: UI component library (MIT License)
6. **SignalR**: Real-time communication library (MIT License)



Consult the individual licenses for detailed terms and conditions.

## Legal Notices

This document was prepared as an account of work sponsored by an agency of the United States government. Neither the United States government nor Lawrence Livermore National Security, LLC, nor any of their employees makes any warranty, expressed or implied, or assumes any legal liability or responsibility for the accuracy, completeness, or usefulness of any information, apparatus, product, or process disclosed, or represents that its use would not infringe privately owned rights.

Reference herein to any specific commercial product, process, or service by trade name, trademark, manufacturer, or otherwise does not necessarily constitute or imply its endorsement, recommendation, or favoring by the United States government or Lawrence Livermore National Security, LLC. The views and opinions of authors expressed herein do not necessarily state or reflect those of the United States government or Lawrence Livermore National Security, LLC, and shall not be used for advertising or product endorsement purposes. recent network connections from chrome.exe and explain any unusual patterns.



### ## Multi-Platform Support

Wintap 7 introduces a foundation for multi-platform support, enabling telemetry collection across Windows, Linux, and MacOS environments.

### ### Platform Abstraction Layer

The platform abstraction is implemented through a hierarchy of base classes:

BaseCollector (abstract)

- |— BaseWinCollector (Windows-specific)
- |— [Future] BaseUnixCollector (Linux/MacOS)

The ``SubscriptionManager`` class handles platform detection and initialization of the appropriate collectors:

```
```csharp
public class
SubscriptionManager
{
    internal void
    Start()
}
```

```

        {
            if
(RuntimeInformation.IsOSPlatform(OSPlatform.Windows))
            {
                winCollectors =
winSubMgr.Start();
            }
            else if
(RuntimeInformation.IsOSPlatform(OSPlatform.Linux))
            {
                linuxCollectors =
linuxSubMgr.Start();
            }
            else if
(RuntimeInformation.IsOSPlatform(OSPlatform.OSX))
            {
                // Mac support future implementation
            }
        else
        {
            WintapLogger.Log.Append("Running on an unsupported platform", LogLevel.Info);
        }
    }
}

```

Windows Implementation

Windows telemetry collection is primarily based on ETW:



csharp

```
internal class WindowsSubscriptionManager
{
    internal List<BaseWinCollector>
Start()
    {
        List<BaseWinCollector> baseCollectors = new List<BaseWinCollector>();

        // Start process collector first for process
        attribution      ProcessCollector pc = new
ProcessCollector();      pc.Start();
        baseCollectors.Add(pc);

        // Start other Windows-specific
        collectors      // ...      return
        baseCollectors;
    }
}
```

Linux Implementation

Linux support is implemented through the LinuxSubscriptionManager :

csharp

```
public class LinuxSubscriptionManager
{
    internal List<BaseCollector>
Start()
    {
        List<BaseCollector> baseCollectors = new List<BaseCollector>();

        // Start process collector
        ProcessCollector pc = new ProcessCollector();
        pc.Start();

        // Start Sysdig collector for system calls
        SysdigCollector sysdig = new SysdigCollector(Path.Combine(
Environment.GetFolderPath(Environment.SpecialFolder.CommonApplicationData),
        "Wintap", "Sysdig", "ygm-class-long-99.json"));
        sysdig.Start();
    }
}
```

```

baseCollectors.Add(pc);
baseCollectors.Add(sysdig);
        return
baseCollectors;
    }
}

```

MacOS Implementation Strategy

For future MacOS support, a similar approach would be used:

```

csharp

public class MacOSSubscriptionManager
{
    internal List<BaseCollector>
Start()
    {
        List<BaseCollector> baseCollectors = new List<BaseCollector>();

        // Implement MacOS-specific collectors
        // Potentially Leverage DTrace, Endpoint Security Framework, or kauth
        return
baseCollectors;
    }
}

```

Implementing Cross-Platform Collectors



To create a truly cross-platform collector:

1. Inherit from the base BaseCollector class:

```

csharp

public class CrossPlatformCollector : BaseCollector
{
    public override bool
Start()
    {
        if
(RuntimeInformation.IsOSPlatform(OSPlatform.Windows))
        {
            // Windows-specific initialization

```

```
        }           else if
(RuntimeInformation.IsOSPlatform(OSPlatform.Linux))
        {
            // Linux-specific initialization
        }           else if
(RuntimeInformation.IsOSPlatform(OSPlatform.OSX))
        {
            // MacOS-specific initialization
        }
return true;
    }
}
```

2. Use platform-specific data sources with a common output format
3. Normalize platform-specific paths and identifiers
4. Handle platform-specific error conditions appropriately

Linux Data Collection Options

For Linux telemetry collection, several options exist:

1. **Sysdig**: Already implemented for system call tracing
2. **eBPF**: Efficient kernel-level tracing
3. **Audit Framework**: Security-focused event collection
4. **Inotify/Fanotify**: File system monitoring
5. **procfs/sysfs**: Process and system information

6. **netlink**: Network monitoring

Example implementation using eBPF: `csharp` 

```
public class EBPFCollector : BaseCollector
{
    private Process
    ebpfProcess;

    public override bool
    Start()
    {
        // Start eBPF program and configure data collection
        ProcessStartInfo psi = new ProcessStartInfo();
        psi.FileName = "bpfttrace";
        psi.Arguments = "-e 'tracepoint:syscalls:sys_enter_* { @[probe] = count(); }'";
        psi.RedirectStandardOutput = true;
        psi.UseShellExecute = false;
        ebpfProcess = Process.Start(psi);
        ebpfProcess.OutputDataReceived += ProcessData;
        ebpfProcess.BeginOutputReadLine();

        return true;
    }

    private void ProcessData(object sender,
    DataReceivedEventArgs e)
    {
        if
        (string.IsNullOrEmpty(e.Data))
        return;

        // Parse eBPF output
        // Create WintapMessage
        // Send to EventChannel
    }
}
```

MacOS Data Collection Options

For future MacOS support, consider these technologies:

1. **Endpoint Security Framework**: Modern API for security events
2. **DTrace**: Comprehensive system tracing
3. **kauth**: Kernel authorization callbacks

4. **fs_usage/sc_usage**: File system and system call monitoring

5. **OpenBSM**: Auditing framework

Example implementation using Endpoint Security: `csharp` 

```
public class EndpointSecurityCollector : BaseCollector
{
    private Process
esProcess;

    public override bool
Start()
    {
        // Start helper process to interface with Endpoint Security
API
        // (Note: This is conceptual as direct C# bindings may not
exist)
        ProcessStartInfo psi = new ProcessStartInfo();
psi.FileName = "wintap-es-helper";
psi.RedirectStandardOutput
= true;
psi.UseShellExecute = false;
esProcess = Process.Start(psi);
esProcess.OutputDataReceived += ProcessData;
esProcess.BeginOutputReadLine();

return true;
    }
    private void ProcessData(object sender,
DataReceivedEventArgs e)
    {
        // Process Endpoint Security events
    }
}
```

API Reference

This section provides detailed reference information for key Wintap APIs.

EventChannel API

`csharp`



```
// Send an event through the system public static
void Send(WintapMessage streamedEvent);
```

```
// Compile and deploy an EPL query public static EPDeployment
compileDeploy(EPRuntime runtime, String epl);

// Manage a workbench query internal static EsperQuery
ManageWorkbenchQuery(EsperQuery q);

// Get all saved workbench queries
internal static List<EsperQuery> getWorkbenchState();
```

StateManager API

csharp



```
// Access static state properties public static
Guid AgentId { get; private set; } public static
bool UserBusy { get; set; } public static string
ActiveUser { get; set; } public static DateTime
LastUserActivity { get; set; } public static bool
OnBatteryPower { get; set; } public static int
PidFocus { get; set; }

// Update Wintap collector settings internal static void
SetWintapSettings(Dictionary<string, bool> settings);

// Get the last boot time internal static
DateTime refreshLastBoot();

// Get the local IP address public
static string GetLocalIpAddress();
```

WintapLogger API

csharp



```
// Log a message with various metadata public void Append(string entry, LogLevel
targetVerbosity, bool logToEventLog = false, EventLogEntryType eventLogType =
EventLogEntryType.Information, int eventId = 1000,
[CallerMemberName] string memberName = "",
[CallerFilePath] string sourceFilePath = "");
```

```
// Close the Log
public void Close();

// Access the Logger instance
public static WintapLogger Log => _instance;
```

Plugin API Interfaces

csharp



```
// Subscribe to Wintap events
public interface ISubscribe
{
    void Subscribe(WintapMessage
eventMsg);    EventFlags Startup();
void Shutdown(); }

// Execute scheduled tasks
public interface IRun
{
    void Run();
    RunManifest
RunStartup();    void
RunShutdown(); }
```

```
// Process query results
public interface IQuery
{
    List<EventQuery> Startup();
void Process(QueryResult result);
void Shutdown();
}
```

REST API Endpoints

Streams API

- GET /api/streams : Get all queries
- GET /api/streams/{name} : Get a specific query
- POST /api/streams : Create or update a query
-

DELETE /api/streams : Delete all queries

Esper Service API

- GET /api/EsperService : Get performance metrics

Tree API

- GET /api/Tree : Get process tree data

LLM API

- PUT /api/LLM/Inference : Submit a query to the AI assistant
- POST /api/LLM/Clear : Clear the conversation history
- POST /api/LLM/Upload : Upload a document to the knowledge base

Wintap Service API

- GET /api/WintapService : Get current configuration
- POST /api/WintapService : Update configuration

Contributing Guidelines

Contributions to Wintap are welcome. This section outlines the process and standards for contributing.

Code Style

Wintap follows these coding standards:

- Use Microsoft's C# Coding Conventions
- Follow the .NET Framework Design Guidelines
- Use meaningful variable and method names
- Include XML documentation comments for public APIs
- Keep methods focused and concise
- Use nullable reference types appropriately
- Add appropriate exception handling

Development Environment Setup

1. Install Visual Studio 2022 or later
2. Install the .NET 6.0 SDK or later
3. Install the Angular CLI for web UI development
4. Clone the Wintap repository
5. Open the solution in Visual Studio
6. Build the solution

Testing Requirements

All contributions should include appropriate tests:

1. **Unit Tests:** Tests for individual components
2. **Integration Tests:** Tests for component interactions
3. **Performance Tests:** For performance-sensitive changes
4. **UI Tests:** For web interface changes

Use MSTest or NUnit for test implementation.

Pull Request Process

1. Fork the repository
2. Create a branch for your feature (`feature/your-feature-name`)
3. Make your changes
4. Add or update tests
5. Ensure all tests pass
6. Update documentation
7. Submit a pull request with a clear description of changes

Code Review Criteria

Pull requests are evaluated based on:

- Adherence to coding standards
- Test coverage

- Performance impact
- Security implications
- Documentation quality
- Compatibility with existing code

Documentation

Update the following documentation when making changes:

- Code comments
- XML API documentation
- README files
- This developer guide
- Release notes

AI Assistant Integration

Wintap 7 integrates an AI assistant that provides intelligent analysis of telemetry data and helps users understand system behavior.

Architecture

The AI integration consists of several components:

1. **LLMController**: ASP.NET Core controller that handles AI requests
2. **InferenceHub**: SignalR hub for real-time AI communication
3. **Semantic Memory**: Vector database for storing contextual information
4. **Chat Interface**: Web UI component for interacting with the AI

Semantic Kernel Integration

Wintap uses Microsoft's Semantic Kernel framework to interact with the AI models:

csharp

```
var aiBuilder = Kernel.CreateBuilder();

// Add the Ollama chat completion
service
aiBuilder.AddOllamaChatCompletion(
modelId: "gemma3:1b",      endpoint: new
Uri(OllamaEndpoint) );

// Add the Ollama embedding service
aiBuilder.AddOllamaTextEmbeddingGeneratio
n(      modelId: EmbeddingModel,
endpoint: new Uri(OllamaEndpoint) );

// Build the kernel with both services
var kernel = aiBuilder.Build();
```

RAG (Retrieval-Augmented Generation) Implementation

The AI assistant uses RAG to provide context-aware responses: csharp



```
// Create memory using the embedding service and SQLite
var sqliteMemoryStore =
SqliteMemoryStore.ConnectAsync(DatabasePath).GetAwaiter().GetResult();

ISemanticTextMemory memory = new MemoryBuilder()
    .WithLoggerFactory(kernel.LoggerFactory)
    .WithMemoryStore(sqliteMemoryStore)
    .WithTextEmbeddingGeneration(embeddingService)
    .Build();
```

This allows the assistant to:

1. Store Wintap documentation and knowledge
2. Retrieve relevant context for queries
3. Provide accurate, contextual responses



AI Request Processing Flow

1. User submits a query through the chat interface
2. The query is sent to the LLMController
3. The controller searches the vector database for relevant context
4. Context is combined with the query and system prompt
5. The combined prompt is sent to the LLM
6. Streaming responses are delivered via SignalR
7. The UI displays the response to the user

csharp

```
[HttpPut("Inference")] public async Task Put([FromBody]
PromptModel promptModel)
{
    // Retrieve relevant information from the semantic memory
    StringBuilder builder = new StringBuilder();
    await foreach (MemoryQueryResult result in memory.SearchAsync(
collectionName, question, 3, minRelevance, withEmbeddings: true))
    {
        builder.AppendLine(result.Metadata.Text);
    }

    // Add context to the chat
    if (builder.Length != 0)
    {
        chat.AddUserMessage("USER QUERY: " + question +
"\n\n" +
        "Retrieved data: " + builder.ToString());
    }

    // Get streaming response    await foreach
(StreamingChatMessageContent message in
ai.GetStreamingChatMessageContentsAsync(chat, settings))
    {
        Inference inf = new Inference() {
            Prompt = question,
            Response = message.Content,
            TokensUsed = 0
        };
        this.hubContext.Clients.All.SendAsync("ReceiveMessage", inf, "OK");    await
    }
}
```

Document Embedding for Knowledge Base

Users can upload documentation to enhance the AI's knowledge:



csharp

```
[HttpPost("Upload")] public async Task<IActionResult>
Upload(IFormFile file)
{
    using (var stream = new
    MemoryStream())
    {
        await file.CopyToAsync(stream);
        stream.Position = 0;
        using (var reader =
        new StreamReader(stream))
        {
            string fileContent = await
            reader.ReadToEndAsync();

            // Split content into manageable chunks
            List<string> lines = TextChunker.SplitPlainTextLines(fileContent,
            128);
            List<string> paragraphs =
            TextChunker.SplitPlainTextParagraphs(
            lines, chunkSize,
            overlapSize, " ");

            // Save each chunk to the vector database
            for (int i = 0; i < paragraphs.Count; i++)
            {
                if
                (!string.IsNullOrEmpty(paragraphs[i]))
                {
                    await
                    memory.SaveInformationAsync(
                    collectionName,
                    paragraphs[i], $"paragraph{i}");
                }
            }
        }
    }
    return Ok();
}
```

System Prompt Configuration

The AI assistant's behavior is guided by a system prompt, which can be configured: csharp



```
string systemPrompt = Settings.Default.SystemPrompt;
ChatHistory chat = new Microsoft.SemanticKernel.ChatCompletion.ChatHistory(systemPrompt);
```

Default system prompt:

You are an AI assistant that analyzes and interprets host-based Windows telemetry and offers guidance to improve performance and security of the system.

Chat Interface

The web UI provides a chat interface for interacting with the AI assistant: typescript



```
@Component({  selector: 'app-chat',
  templateUrl: './chat.component.html',
  styleUrls: ['./chat.component.scss']
}) export class ChatComponent implements
OnInit {  messages: ChatMessage[] = [];
  newMessage: string = '';  isLoading:
  boolean = false;

  // AI interaction
  methods  sendMessage() {
  // Send message to AI
    // Display streaming response
  }

  // Document upload for knowledge base
  uploadDocument(event: any) {
    // Handle file upload
  }
}
```

AI Assistant Capabilities

The AI assistant can help with:

1. **Telemetry Analysis:** Examining patterns in collected data
2. **Query Generation:** Creating EPL queries for specific scenarios
3. **Troubleshooting:** Identifying potential issues from telemetry
4. **Documentation:** Explaining Wintap concepts and features



Example queries:



Analyze the